Human Inspired AI for Intelligent Traffic Management

IT 207 Project Report | Seven chapter submission

Student details: to be completed by student

Evidence note: measured results in this report come from the seeded software simulator. No real traffic video, labels, trained weights, or SUMO run was available in the supplied workspace.

Abstract

This project continues an existing Python prototype with detection and tracking modules, lane analysis, predictor implementations, signal controllers, a software simulator, API, and dashboard. The simulator comparison was repaired and evaluated under four synthetic demand scenarios. AI-assisted control improved both delay and throughput in the uneven scenario but did not outperform fixed timing in medium or heavy balanced traffic. This single seeded run does not establish a general performance gain. Video models and SUMO could not be run because required model files, input data, and runtime dependencies were absent.

Chapter guide

Chapter 1 Introduction

Chapter 2 Human Intelligence Theory

Chapter 3 Literature Review

Chapter 4 Proposed Methodology

Chapter 5 Implementation

Chapter 6 Results and Evaluation

Chapter 7 Ethics Conclusion and Future Work

Chapter 1 Introduction

Urban intersections experience uneven demand, and a fixed allocation can leave a busy approach waiting while a quiet approach receives green time. This project continues an existing Python prototype that uses video-based perception and software signal simulation to explore that problem. Its purpose is to connect selected human cognition ideas to measurable AI components, build a reproducible software demonstration, and assess adaptive control against a fixed-time baseline.

The project scope is software-only. Recorded video is the intended input; no physical sensors, embedded devices, or live traffic-light control are used. The supplied workspace includes detection, tracking, analysis, prediction, optimization, a FastAPI backend, and a Streamlit dashboard. At audit time it did not include a traffic video, labeled data, model weights, trained predictors, or prior experiment results. Consequently, this report distinguishes implemented code from features that could not be executed in this environment.

Objectives are to inspect and preserve the existing architecture; repair the fallback signal simulation and experiment comparison; retain the YOLO and ByteTrack video pipeline; implement lane metrics and short-term prediction interfaces; compare three signal controllers; and document ethical and technical limitations. Experimental results in this submission come only from the seeded software simulator, not from road observations or labeled video.

Chapter 2 Human Intelligence Theory

The design uses human cognition as an engineering analogy, not as a claim that the software reproduces human intelligence. Selective attention is represented by configurable lane and region-of-interest polygons. The detector processes vehicle classes of interest and the analyser aggregates vehicles by lane, focusing computation on relevant parts of a frame. This is a task-specific filter; it can miss vehicles outside incorrectly configured regions.

Working memory is represented by a bounded history buffer and persistent SQLite observations. Recent lane counts, queues, and traffic states provide context for prediction and control. The buffer is an external software record, not a psychological measurement. Learning is represented by the Random Forest, XGBoost, and LSTM training code and lagged traffic features. No trained model or validated prediction result was supplied, so the learning component remains an implementation capability rather than an experimentally established result.

Decision-making is represented by controller logic that assigns green duration from demand, queue, predicted count, waiting urgency, and fairness weights, subject to configured bounds. The fixed-time controller provides a baseline; the rule-based and weighted adaptive controllers provide alternatives. These mappings are inspired by selective attention, working memory, learning, and decision-making theories. They are computational proxies only and do not measure people or establish biological equivalence.

Chapter 3 Literature Review

Single-stage YOLO detection frames object localization and classification as one neural-network inference pass [4]. The project config selects Ultralytics YOLO11n and COCO vehicle classes (car, motorcycle, bus, truck). The model weights are not present in the workspace, and detector performance was therefore not measured. ByteTrack associates high- and lower-confidence detections with tracklets to preserve identities through partial occlusion [5]. The project wraps the Ultralytics tracking interface, but there is no annotated video or ground-truth track file for IDF1 or MOTA evaluation.

Traffic signal research commonly compares fixed timing with demand-responsive control using simulation. SUMO is an open-source microscopic traffic simulator with interfaces for traffic-light control [6]. Although this project contains a SUMO integration module, SUMO and TraCI were not installed, and the generated network does not contain a verified signal phase plan. The included experiment therefore uses the local queue simulator and is labelled accordingly.

The additional repository by Prit2341 describes density-based green allocation and coordinating consecutive signals, and its README claims a 35 percent throughput improvement [7]. The claim is reported by that repository and is not reproduced here. No code was copied. The repository tree did not expose a license file during inspection, so only high-level ideas were considered. A remaining research gap for this project is evaluating corridor coordination on a valid multi-intersection network with calibrated demand, turning ratios, startup delays, and measured travel time.

Chapter 4 Proposed Methodology

The intended video path is: recorded video frame; YOLO detection; ByteTrack association; ROI-based lane assignment; lane counts and class totals; density, queue, and state estimates; database history; optional forecasts; then signal-duration recommendations. The dashboard exposes video analysis, lane monitoring, prediction, signal optimization, and experiment views. SQLite persists observations and decisions. A detailed component diagram is included as architecture.png.

The controller experiment uses four approaches and three controllers: fixed-time with 30 seconds of green per approach, rule-based demand scoring, and a weighted AI-assisted controller. Each scenario runs for 1,800 simulation seconds using seeded Poisson vehicle arrivals. The low, medium, and heavy scenarios split their configured total flow equally among approaches. The uneven scenario uses 55, 25, 15, and 5 percent shares. Each controller receives the same random seed and approach arrival rates within a scenario. Saturation flow is set to 0.5 vehicles per green second, and an assumed 5 metres per queued vehicle converts queue counts to an indicative distance.

Reported outputs are mean simulated queue delay per completed departure, mean queue length in metres per approach, throughput in vehicles per hour, and fraction of lane-seconds above the configured moderate threshold. These are outputs of a simplified queue model. They are not travel-time measurements, physical queue observations, or SUMO measurements. Each scenario-controller pair is a single seeded run; uncertainty and multi-seed confidence intervals are not estimated.

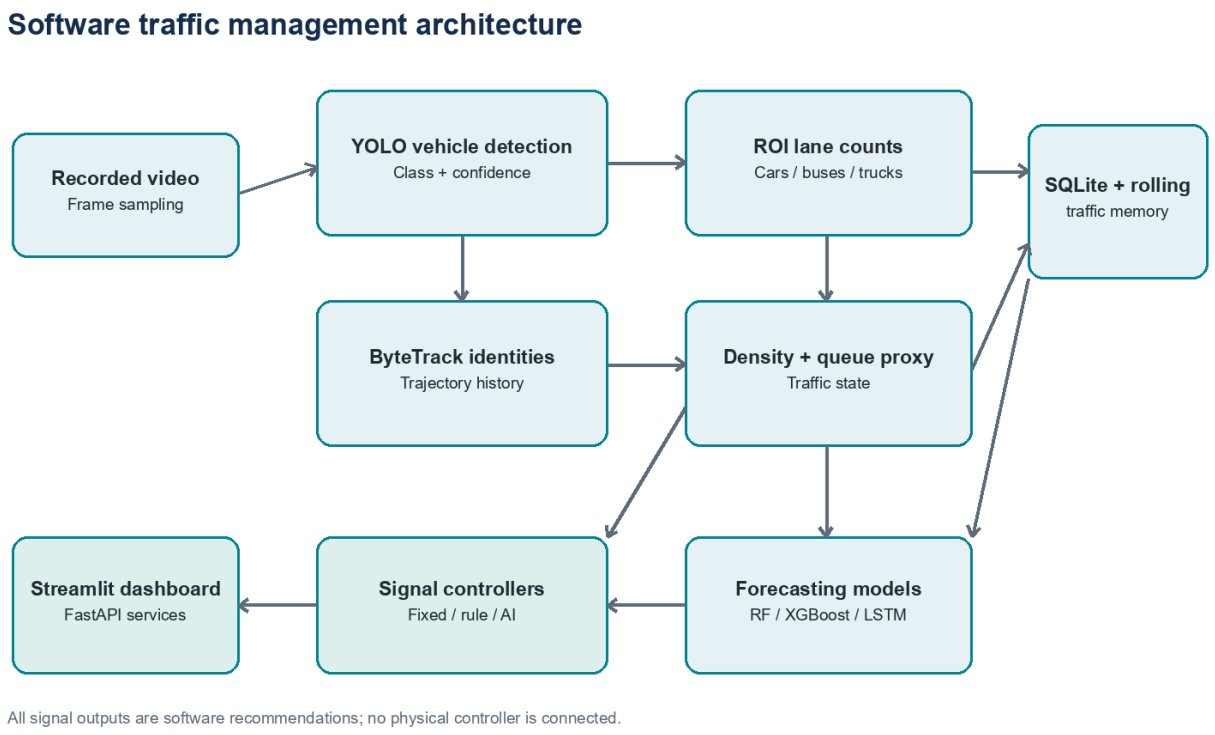


Figure 1. Software architecture. Signal outputs are software recommendations only.

Chapter 5 Implementation

The existing source tree was retained. Core modules are src/detection/detector.py, src/tracking/tracker.py, src/analysis/traffic_analyser.py, src/prediction/predictor.py, src/optimization/signal_controller.py, and src/simulation/sumo_runner.py. FastAPI endpoints are in backend/main.py and the seven-page Streamlit dashboard is in dashboard/app.py. Configuration and thresholds are in config.yaml. Detection uses the configured YOLO11n name, confidence threshold, and class IDs; tracking uses ByteTrack; the analyser estimates queue length as five metres per vehicle and stores recent history.

The simulator previously treated total cycle count as equal experimental duration, accumulated waiting time repeatedly, and used the same fallback arrival rate for every scenario. It now advances each approach phase over

seeded one-second arrivals, tracks queued vehicle-seconds, uses a common simulated time horizon for controller comparisons, respects the configured scenario rates, and writes an explicitly labelled software_simulation result table. The utility module now imports OpenCV only in video-specific functions, allowing non-video simulation modules to run when OpenCV is unavailable.

The present runtime had Python 3.14 and lacked the project's primary dependencies, video data, model files, and SUMO. A bundled Python 3.12 runtime with NumPy, Pandas, PyYAML, and Loguru was used for the simulator and tests; OpenCV, PyTorch, Ultralytics, Streamlit, and FastAPI were unavailable. The project requirements file specifies Python 3.11. Full dashboard, video, API, prediction-model, and SUMO execution was not verified here.

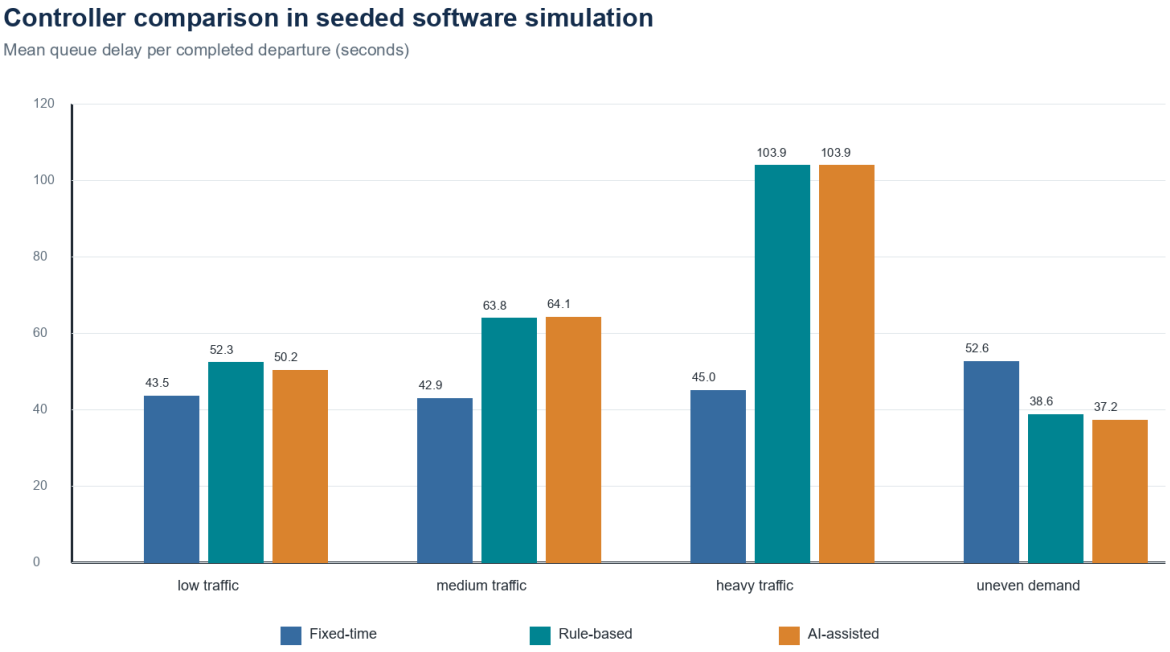
Chapter 6 Results and Evaluation

Table 1 summarizes the measured software simulation results. Values are means from one deterministic seeded run per scenario and controller. Throughput is normalized to vehicles per hour. Queue length is averaged per approach and multiplied by the assumed 5 metres per vehicle. A lower queue delay is preferable; higher throughput is preferable, subject to the limitations of the queue model.

In low traffic, the AI-assisted controller produced 50.22 seconds mean queue delay and 224 vehicles per hour, compared with 43.50 seconds and 218 vehicles per hour for fixed-time. In medium traffic, AI-assisted produced 64.12 seconds and 598 vehicles per hour, versus 42.86 seconds and 606 vehicles per hour for fixed-time. In heavy traffic, AI-assisted produced 103.87 seconds and 1,168 vehicles per hour, versus 45.01 seconds and 1,208 vehicles per hour for fixed-time. In uneven demand, AI-assisted produced 37.19 seconds and 724 vehicles per hour, versus 52.62 seconds and 686 vehicles per hour for fixed-time. Thus the proposed controller did better on both reported measures in the uneven scenario and worse on both measures in the medium and heavy scenarios. The results do not support a general congestion-reduction claim.

Rule-based and AI-assisted scores coincide in balanced cases when their inputs and resulting allocations are equal or similar. Differences in the uneven scenario reflect adaptation to lane demand and waiting urgency. The software tests confirm controller bounds, repeatability for a fixed seed, duration accounting, and arrival/departure consistency. There are no labeled video results, so precision, recall, mAP, IDF1, MOTA, classification F1, forecast MAE, and forecast RMSE are not available.

Scenario	Controller	Delay s	Queue m	Throughput/h
low traffic	Fixed-time	43.5	3.29	218.0
low traffic	Rule-based	52.26	4.06	224.0
low traffic	AI-assisted	50.22	3.91	224.0
medium traffic	Fixed-time	42.86	9.02	606.0
medium traffic	Rule-based	63.79	13.25	598.0
medium traffic	AI-assisted	64.12	13.31	598.0
heavy traffic	Fixed-time	45.01	18.88	1208.0
heavy traffic	Rule-based	103.87	42.12	1168.0
heavy traffic	AI-assisted	103.87	42.12	1168.0
uneven demand	Fixed-time	52.62	12.53	686.0
uneven demand	Rule-based	38.65	9.53	710.0
uneven demand	AI-assisted	37.19	9.35	724.0



Chapter 7 Ethics Conclusion and Future Work

Recorded traffic footage can contain identifiable people, plates, and travel patterns. A deployment should minimize retention, restrict access, blur identifying details where practical, and document consent and lawful purpose. Detection errors may vary with camera angle, weather, lighting, vehicle size, and local vehicle mix. Miscounts can shift green allocation unfairly; the controller should enforce minimum service and waiting-time limits and expose its recommendations for human review.

The current prototype is not validated for public-road control. Queue length and speed are approximate without camera calibration. The simulator omits pedestrians, turning conflict, startup loss, lane changing, spillback, network geometry, and calibrated distributions. The SUMO route/network integration has not been validated. In particular, a safe signal phase plan and conflict-free movements must be confirmed before treating SUMO results as evidence. The project does not connect to or command physical equipment.

The project demonstrates a coherent software architecture and a reproducible local controller experiment, but it does not establish that the AI-assisted policy is superior in general. The next steps are to install the supported Python runtime and dependencies, provide licensed video and annotations, validate model paths, measure detection and tracking metrics, train forecasting models on time-ordered real data, run repeated scenarios with confidence intervals, and validate multi-intersection coordination in SUMO.

References

[1] IT 207 Human Intelligence Project Details. Supplied course authority PDF, 6 pages. Seven-chapter outline, assessment rubric, and recommended submission package.

[2] Broadbent, D. E. (1958). Perception and Communication. Pergamon Press.

[3] Baddeley, A. (2000). The episodic buffer: a new component of working memory? Trends in Cognitive Sciences, 4(11), 417-423. https://doi.org/10.1016/S1364-6613(00)01538-2

[4] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. CVPR, 779-788. https://openaccess.thecvf.com/content_cvpr_2016/html/Redmon_You_Only_Look_CVPR_2016_paper.html

[5] Zhang, Y. et al. (2022). ByteTrack: Multi-Object Tracking by Associating Every Detection Box. ECCV. https://www.ecva.net/papers/eccv_2022/papers_ECCV/papers/136820001.pdf

[6] Eclipse SUMO documentation, SUMO at a Glance. https://sumo.dlr.de/docs/SUMO_at_a_Glance.html

[7] Prit2341. Traffic-Management-Using-AI, README and simulation files. https://github.com/Prit2341/Traffic-Management-Using-AI (accessed 2026-11-02). No source code reused; repository-reported 35% result treated as unverified.